

Super Sampling Project Report

Introduction

Supersampling is an anti-aliasing technique that improves image quality by sampling multiple points within a pixel and averaging the results to produce smoother transitions and reduce visual artifacts. In this project, we extended the existing codebase by implementing multiple types of supersampling, including both methods applied during the ray tracing process and techniques for post-processing. These additions allow for comparisons between uniform sampling, jittered sampling, and filtering approaches like box and Gaussian filtering, enhancing flexibility and performance in image rendering.

Overview

The goal of this project is to implement and compare multiple supersampling techniques to improve the visual quality of rendered images in a ray tracer. Supersampling, as an anti-aliasing technique, reduces visual artifacts such as jagged edges by taking multiple samples within a pixel and averaging them. This project incorporates supersampling at two stages: during the tracing process and in post-processing. These stages allow for a side-by-side comparison of different sampling and filtering approaches.

During tracing, both uniform sampling and jittered sampling were implemented. Uniform sampling divides each pixel into a grid, taking samples at regular intervals, while jittered sampling introduces small random offsets to the sampling positions to add variation and reduce regular sampling artifacts. Post-processing, on the other hand, utilizes uniform sampling to refine the output image further by resampling and filtering pixel data.

The flexibility provided by these implementations allows for an in-depth comparison of how different combinations of sampling and filtering impact the final image quality. The project demonstrates how varying approaches can balance between simplicity, computational cost, and visual fidelity.

Algorithms and Techniques

This project incorporates several key algorithms and techniques to achieve super sampling, each with unique strengths and purposes.

Gaussian filtering applies a weighted approach where samples closer to the pixel center contribute more to the final color. The equation we used for the gaussian kernel is

$G(x, y; \sigma) = (1/2\pi\sigma^2)e^{-(x^2+y^2)/2\sigma^2}$ and we got this from page 127 from the textbook *Computer Vision: Algorithms and Applications 2nd Edition* by Richard Szeliski. The x and y is the length and height of the kernel and the sigma is the intensity of the gaussian kernel. This technique

prioritizes smoother transitions and is implemented both during the tracing process with jittered sampling and in post-processing with uniform sampling. The Gaussian kernel ensures that the influence of each sample follows a bell-curve distribution.

Box filtering, a simpler alternative, averages all sampled values equally, giving each sample the same weight. It serves as a baseline for comparison and is implemented in the same contexts as Gaussian filtering: during tracing with jittered sampling and in post-processing with uniform sampling. While less sophisticated than Gaussian filtering, box filtering provides consistent results with lower computational complexity.

Uniform sampling takes evenly spaced samples within a pixel, ensuring complete coverage of the pixel's area. It is employed both during tracing and in post-processing to provide a regular and structured sampling approach. This method is particularly effective in achieving consistent results, especially when paired with filtering techniques.

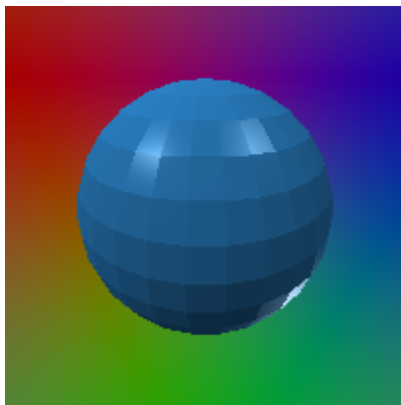
Jittered sampling introduces randomness to the sample positions within a pixel grid during the tracing process. By perturbing the sampling offsets with small random values, this method breaks up regular patterns and mitigates aliasing artifacts caused by uniformity. Jittered sampling is particularly useful for adding realism to rendered edges and transitions.

We also implemented down sampling, where it does the opposite and reduce the render size from the original.

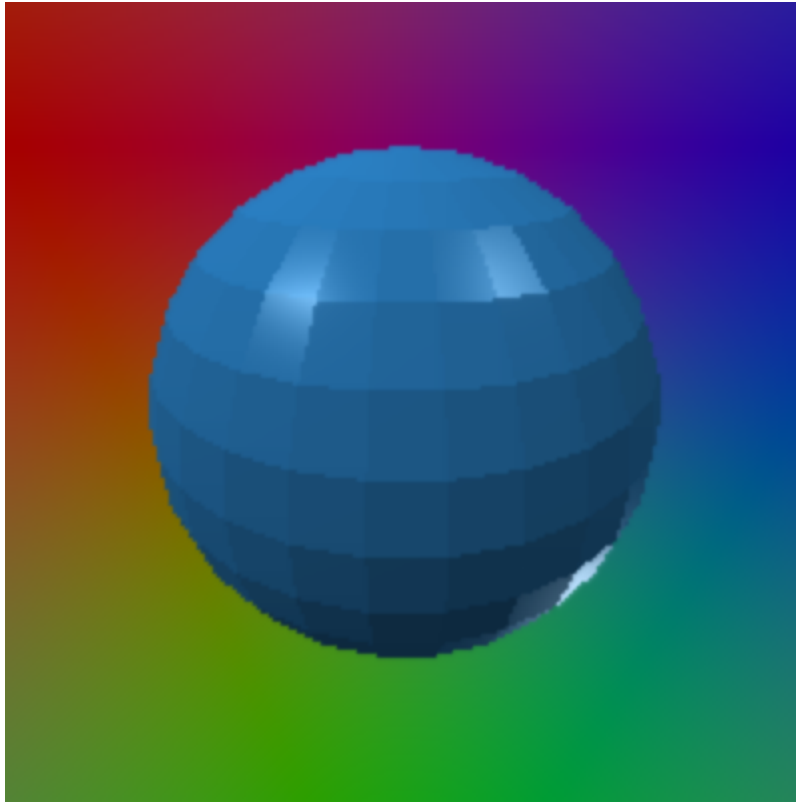
These techniques, when combined, allow for a flexible and comprehensive exploration of the trade-offs involved in anti-aliasing. Each approach offers unique benefits, providing a diverse set of tools to enhance rendered image quality.

Performance

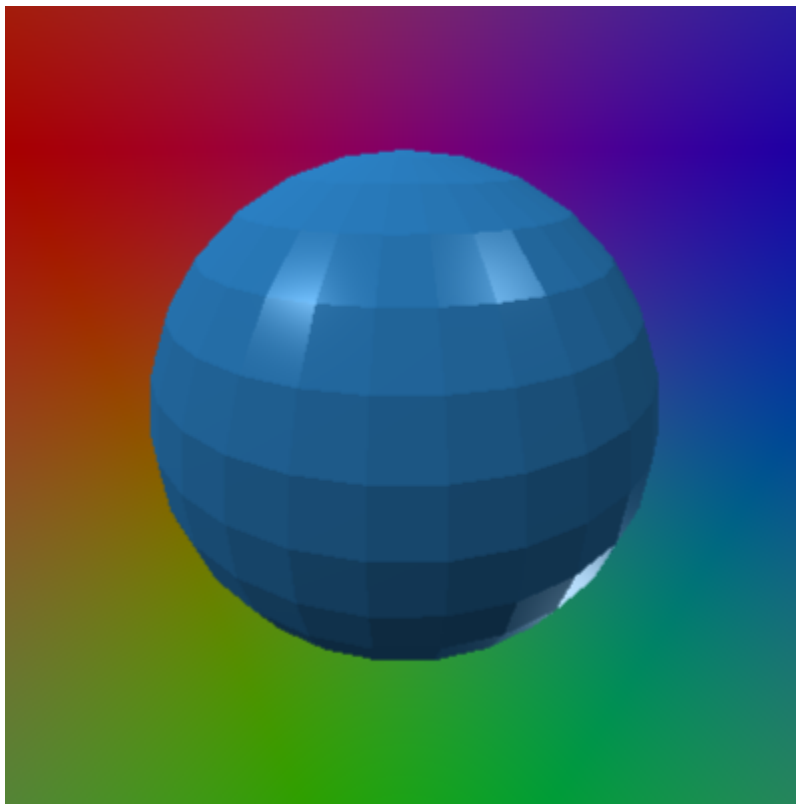
images 200x200 blue spheres.obj



(original)

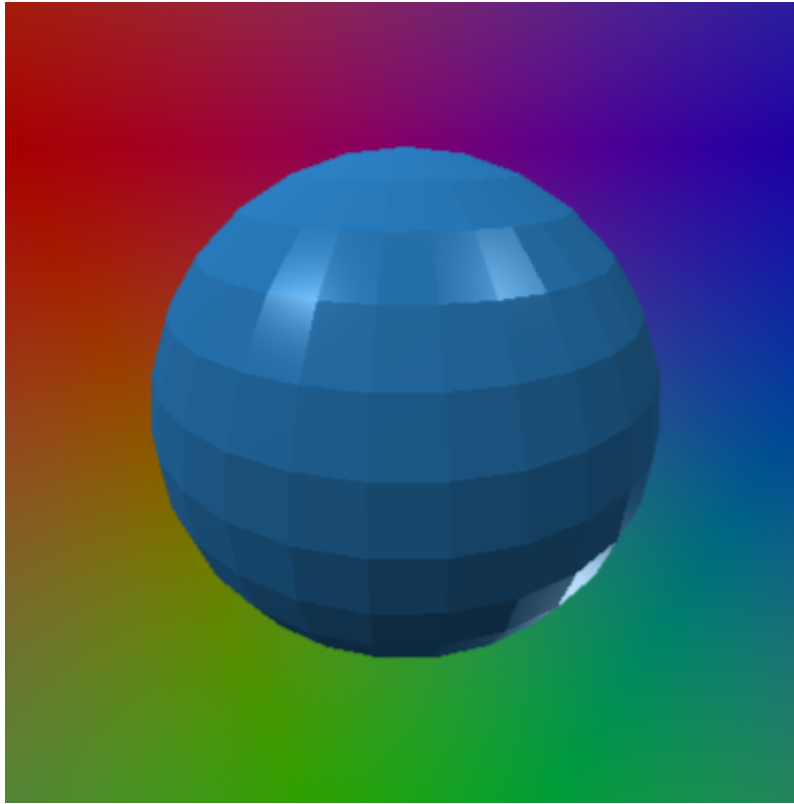


(Box filter anti-aliased image: Applies anti-aliasing to the image after ray tracing (box filter))

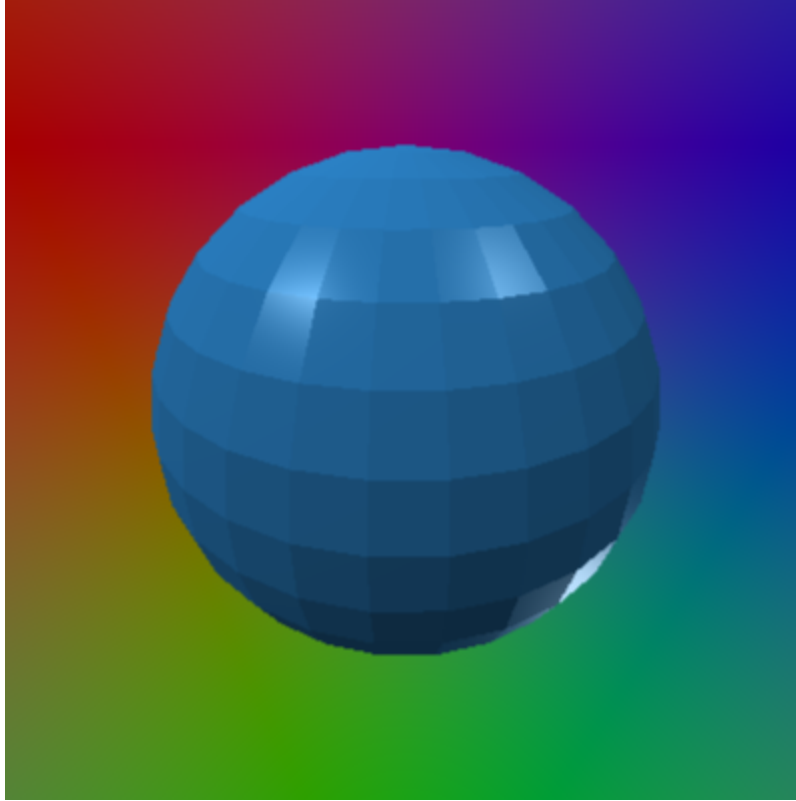


Abe Gizaw, Brian Pascente

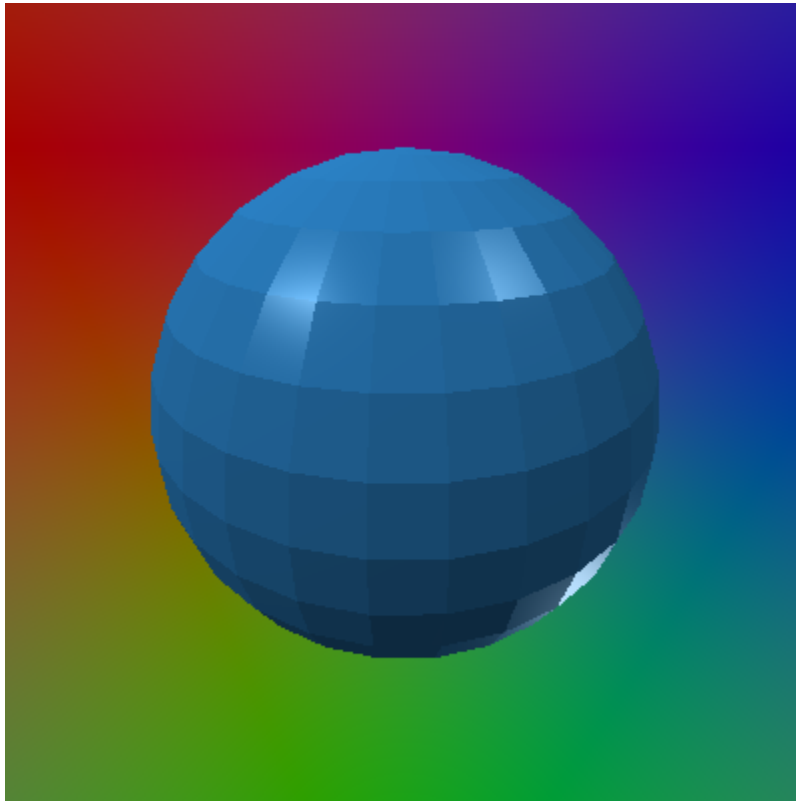
(Per-pixel box filter anti-aliased image: Applies anti-aliasing to the image w/uniform sampling and box filter)



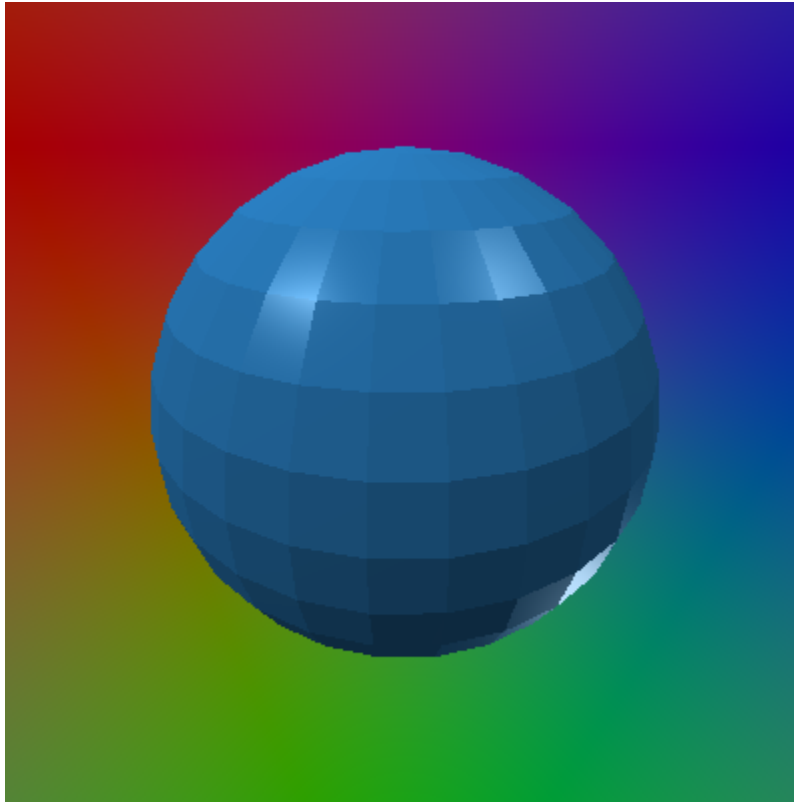
(Per-pixel Gaussian filter anti-aliased image: Applies anti-aliasing to the image w/uniform sampling and Gaussian filter (3x3 kernel with $\sigma=1$))



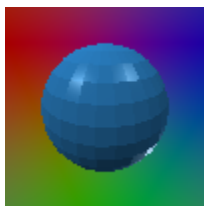
(Per-pixel Gaussian filter anti-aliased image: Applies anti-aliasing to the image w/uniform sampling and Gaussian filter (9x9 kernel with $\sigma=3$))



(Jittered box filter anti-aliased image: Applies anti-aliasing to the image w/jittering sampling and box filter)



(Jittered Gaussian filter anti-aliased image: Applies anti-aliasing to the image w/jittering sampling and Gaussian filter)



(Downsampled image: Downsampled image.)

Conclusion

This project successfully implements and compares/contrasts multiple supersampling techniques. By integrating uniform sampling, jittered sampling, Gaussian filtering, and box filtering at both the tracing and post-processing stages, we provide a flexible framework for analyzing trade-offs and differences between them. The results highlight the strengths and limitations of each approach.

Future work can extend this project by incorporating additional supersampling techniques. For an easier or intermediate challenge, one could implement triangle filtering, which improves weighting across pixels, or explore stratified sampling and Poisson disk sampling to achieve better sample distribution. For more advanced improvements, adaptive sampling could be introduced to focus computational effort on high-contrast areas, and machine learning-based anti-aliasing could be explored for intelligent sampling and reconstruction. These enhancements would further refine the anti-aliasing process, making the system even more efficient and capable of producing higher-quality images.